

A PROJECT REPORT
ON
“Breast Cancer Classifier Web App”

Submitted to
KIIT Deemed to be University

In Partial Fulfilment of the Requirement for the Award of

BACHELOR’S DEGREE IN
INFORMATION TECHNOLOGY

BY

TANMAY ARYA 2006246
APARAJITA JHA 2006008

UNDER THE GUIDANCE OF
SAMPRITI SOOR



SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA - 751024
May 2023

KIIT Deemed to be University

School of Computer Engineering
Bhubaneswar, ODISHA 751024



CERTIFICATE

This is to certify that the project entitled

“Breast Cancer Classifier Web App”

submitted by

TANMAY ARYA

2006246

APARAJITA JHA

2006008

This is a record of bonafide work carried out by them, in the partial fulfillment of the requirement for the award of Degree of Bachelor of Engineering (Information Technology) at KIIT Deemed to be University Bhubaneswar. This work is done during the year 2022-2023, under our guidance.

Date: 04/05/2023

Sampriti Soor
Project Guide

Acknowledgments

We are profoundly grateful to Sampriti Soor for his expert guidance and continuous encouragement throughout to see that this project rights its target from its commencement to its completion.

Sir gave us access to his invaluable industry knowledge, which enabled us to better comprehend the project's specifications. He imparted his expertise and experience to us, allowing us to make well-informed choices and follow the proper course of action throughout the project.

Sir's inspiration and drive helped to keep us on track and committed to our objectives. His upbeat outlook and encouragement helped us stay motivated and get beyond whatever difficulties we ran into while working on the project.

TANMAY ARYA

APARAJITA JHA

Abstract

The objective of this project is to create a machine-learning web application to categorise breast cancer tumours as benign or malignant using the Python libraries Scikit-learn, Pandas, Seaborn, and Streamlit. The programme utilises the Random Forest Classifier method and the Breast Cancer dataset from Scikit-learn to categorise tumours based on certain criteria.

To choose features and view results, the programme offers a user-friendly interface. The programme divides the data into training and testing sets and creates a Random Forest Classifier model once the user selects the attributes they wish to utilise for classification from a sidebar. The programme then makes a prediction for the test data's target variable, and the main interface shows the accuracy score, confusion matrix, and sample forecasts.

This study demonstrates the effectiveness of machine learning and its capacity to correctly categorise difficult data. Users may interact with the application's interactive experience to study and analyse the Breast Cancer dataset using cutting-edge technologies. Overall, this experiment shows how machine learning can be used in healthcare and emphasises how important it is to use technology to progress medical research.

Contents

Section	Page Numbers
Introduction	1
Basic Concepts/ Tools Used	2
Problem Statement / Requirement Specifications	3-6
- Project Planning	6
- System Architecture / Block Diagram	6-8
Implementation	10-13
Standards Adopted	14-15
Conclusion and Future Scope	16
References	17
Individual Contribution	18-19
Plagiarism Report	20

Chapter 1

Introduction

One of the most prevalent cancers in the world, breast cancer significantly affects the health and happiness of millions of individuals. Breast cancer is the most prevalent cancer among women, according to the World Health Organisation, and it is predicted that 1 in 8 women will have it at some point in their lifetime. But men can also develop breast cancer; less than 1% of all cases of breast cancer occur in men. The development of machine learning algorithms has shown considerable promise in increasing the accuracy of breast cancer diagnosis. Early detection and accurate diagnosis are essential for the effective treatment of breast cancer.

"Machine learning algorithms can be used to improve the accuracy of breast cancer diagnosis, which is crucial for successful treatment."

In this project, we'll create a machine learning web application that categorises breast tumours as benign or malignant using the Scikit-learn toolkit. We'll make use of the Scikit-learn Breast Cancer dataset, which provides details on the size, texture, and other characteristics of breast tumours in addition to their diagnosis. The project's objective is to develop an intuitive online interface that makes it simple for users to interact with the model and get precise predictions, as well as to train a machine learning model that can reliably categorise breast tumours based on these attributes.

Breast cancer is a serious public health concern, and successful treatment depends on an accurate diagnosis. Breast cancer diagnosis has historically depended on labor-intensive physical evaluation of biopsy samples by skilled pathologists, which is vulnerable to inter-observer variability. Machine learning algorithms are an appealing approach for enhancing breast cancer diagnosis because they can quickly and correctly analyse vast volumes of data. Additionally, the advancement of machine learning models that can categorise breast tumours using non-invasive imaging methods like mammography has the potential to revolutionise breast cancer detection and diagnosis.

In conclusion, the development of a machine learning web application for diagnosing breast cancer has the potential to increase diagnostic precision and eventually enhance patient outcomes.

Chapter 2

Basic Concepts/ Tools Used

Streamlit:- An open-source Python framework called Streamlit is used to build interactive web apps for data science and machine learning. It eliminates the requirement for HTML, CSS, or JavaScript and enables developers to write just a few lines of code to design user interfaces for their machine learning models.

Pandas:- Pandas is an open-source Python toolkit for manipulating and analysing data. It offers robust data structures, such DataFrames, for working with tabular data and enables developers to quickly clean, filter, and convert data..

Seaborn:- A Matplotlib-based Python data visualisation package is called Seaborn. It offers a more advanced interface for producing statistical visualisations that are more visually beautiful and educational than those created by Matplotlib, such as heatmaps and regression plots.

Scikit-learn:- A well-known machine learning library for Python is called Scikit-learn. It offers a wide range of tools, such as classifiers, regressors, and clustering algorithms, for developing and assessing machine learning models.

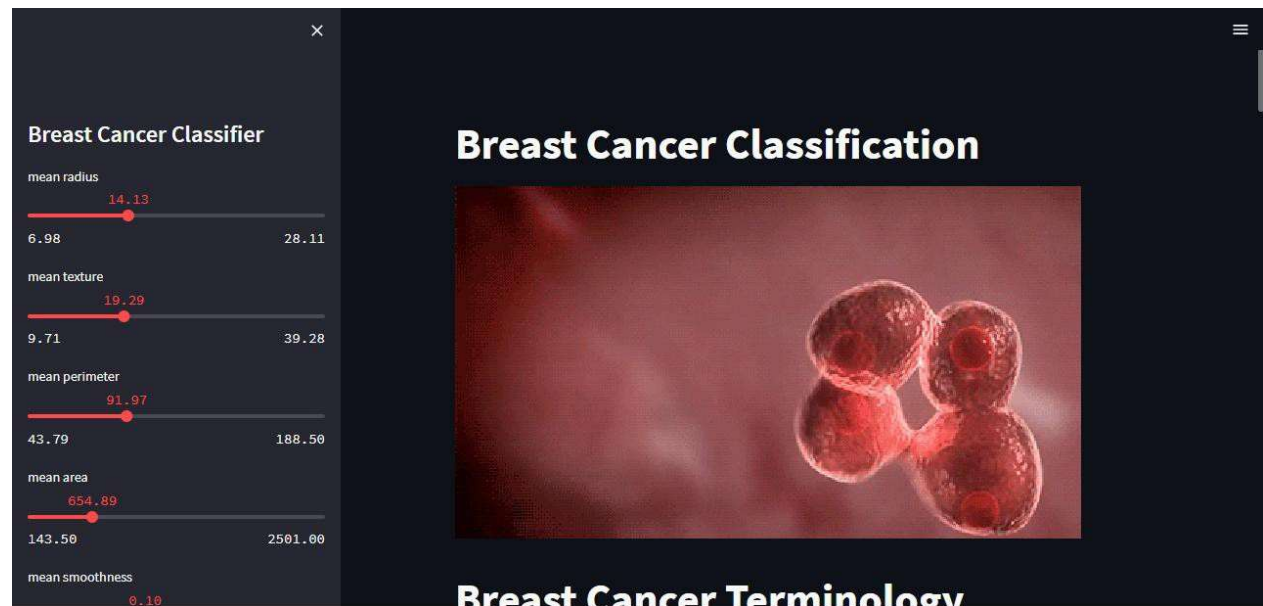
Breast Cancer Dataset:- Scikit-learn offers a well-known dataset called the Breast Cancer dataset. It comprises 30 attributes characterising the properties of the tissue and 569 samples of breast tissue, each of which is classified as benign or cancerous.

Random Forest Classifier:- Popular machine learning algorithm Random Forest is a member of the class of ensemble learning techniques. At training time, a large number of decision trees are built, and the output class is either the mean prediction (regression) or classification of all the individual trees.

Chapter 3

Problem Statement/ Requirement Specifications

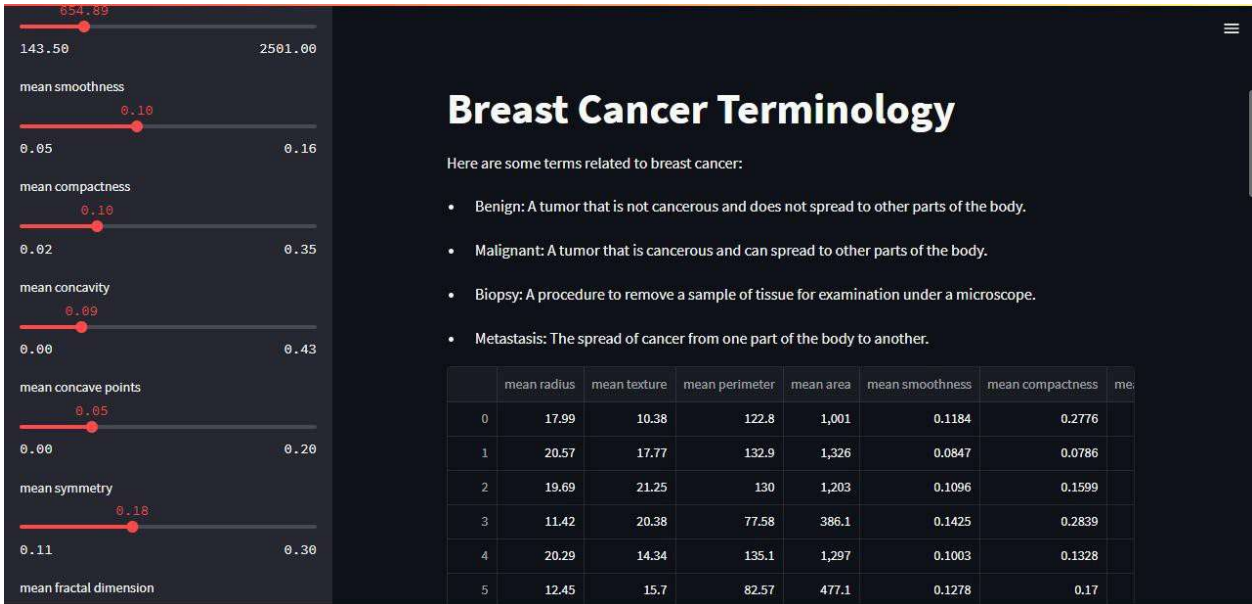
Data Source: Scikit-learn's Breast Cancer dataset was utilised for this research. 30 attributes in this dataset explain the properties of the cell nuclei visible in digital pictures of breast mass biopsies. The target variable is a binary distinction between benign and malignant tumours.



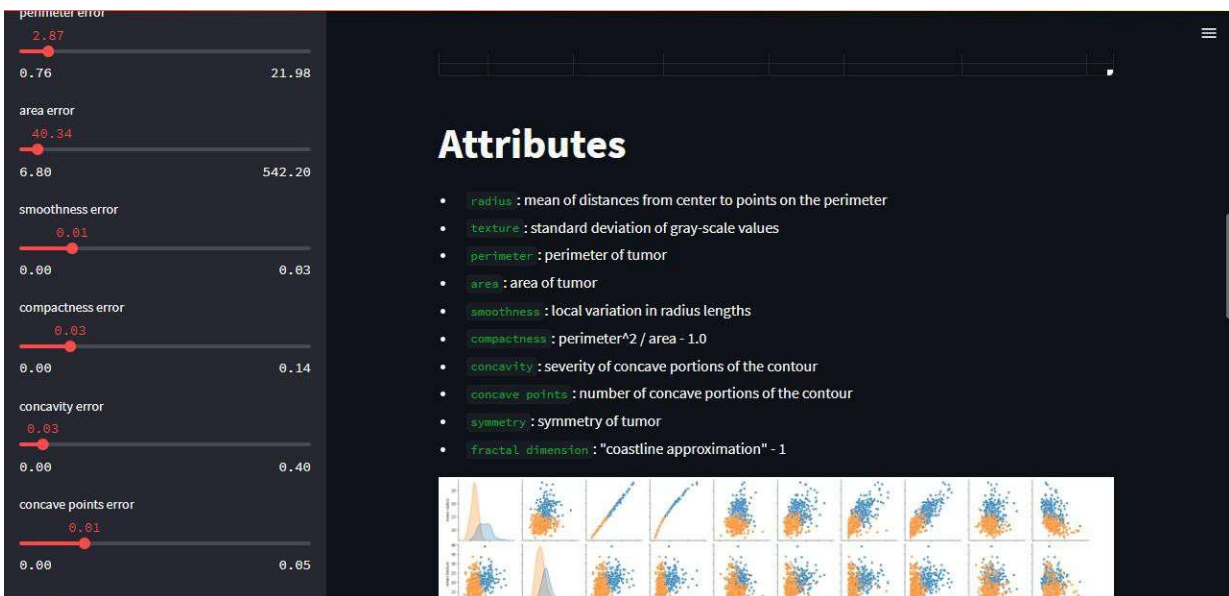
Python and the Streamlit library will be used to implement the aforementioned specifications. The user may choose the features they wish to utilise for categorization in the sidebar of the online application. The accuracy score, confusion matrix, and sample predictions will all be displayed in the main interface. The app will be set up on a web server so that any device with an internet connection may access it.

Medical experts who wish to swiftly categorise breast cancer tumours based on their features are the app's target users. Medical personnel may diagnose patients faster and with more accuracy by utilising this app. The software may also be used as a teaching tool for researchers and students who wish to learn how machine-learning algorithms can be applied to medical datasets.

Feature Selection: In the web app's sidebar, the user may choose which features to utilise for classification. The parameters of the cell nucleus, such as radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension, are the mean, standard error, and worst (mean of the three biggest values).



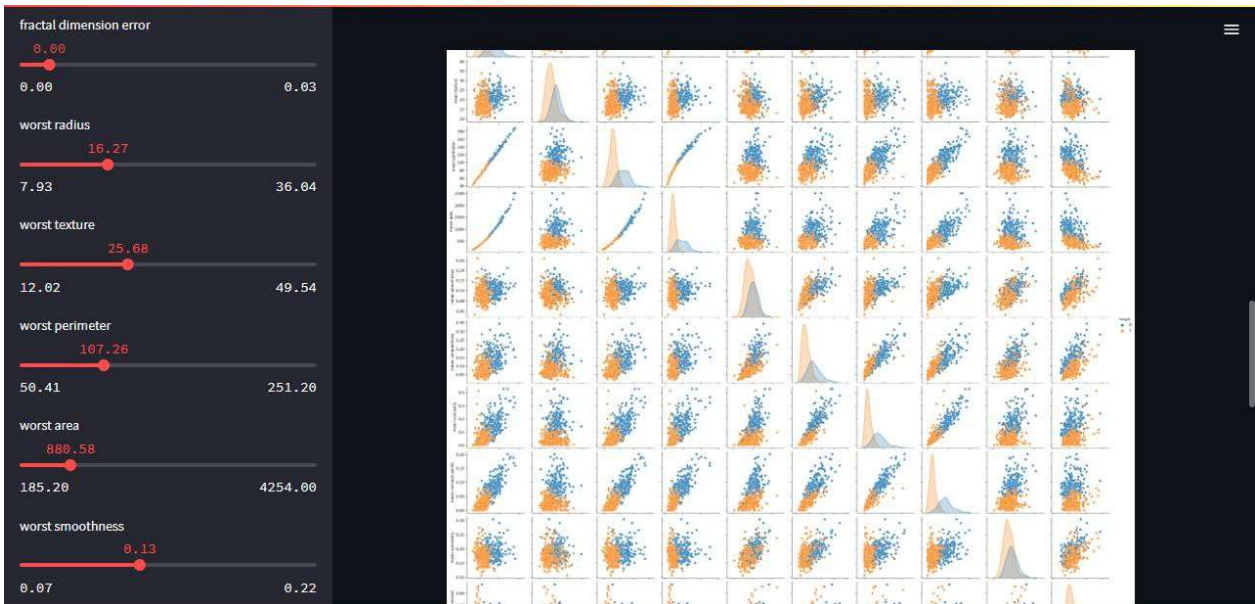
Data Splitting: A 70/30 split will be used to divide the dataset into training and testing sets. The testing set will be used to gauge the model's accuracy, while the training set will be used to fit the model.



Random Forest Classifier: The random forest classifier from Scikit-learn is the classification model that was employed. This algorithm uses an ensemble approach to merge different decision trees in order to get predictions that are more accurate.

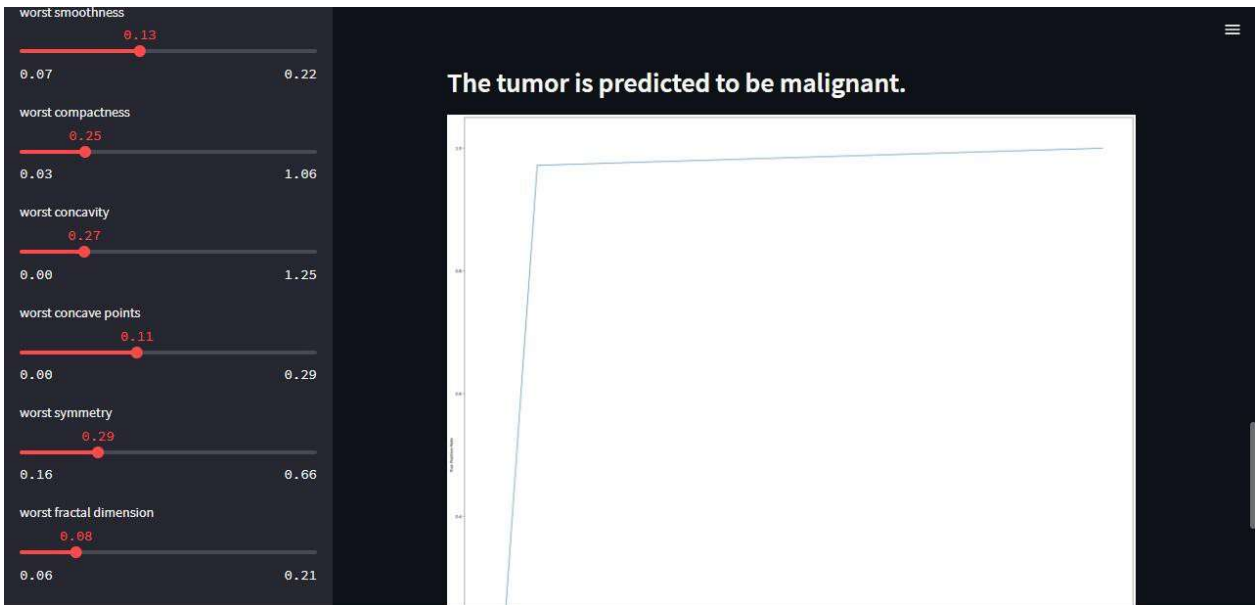
Accuracy Score: The accuracy score is a statistic used to assess how well the model is working. The percentage of tumours in the testing set that were properly categorised is the accuracy score.

Confusion Matrix: To illustrate the number of benign and malignant tumours that were properly and wrongly identified, a confusion matrix will be shown in the main app.

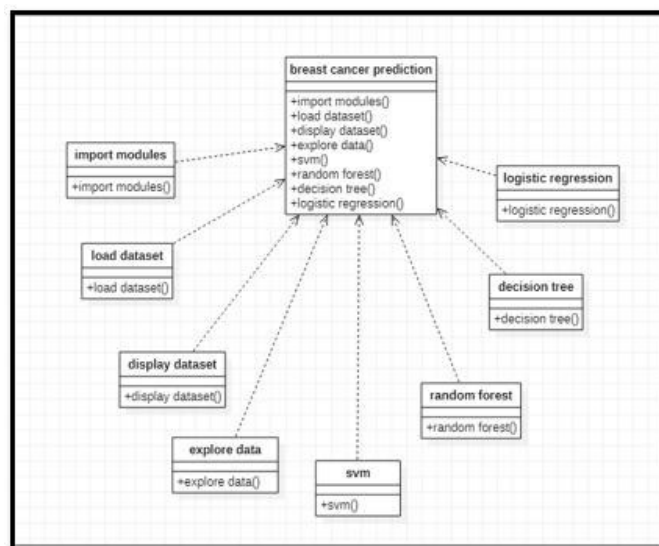


There will be four items in the confusion matrix, which will be shown in the main app: true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP). The number of benign tumours that were appropriately identified as benign is represented by the true negatives. The amount of benign tumours that were mislabeled as malignant is represented by the false positives. The number of malignant tumours that were mistakenly labelled as benign is shown by the false negatives. The number of malignant tumours that were appropriately identified as such is reflected in the true positives.

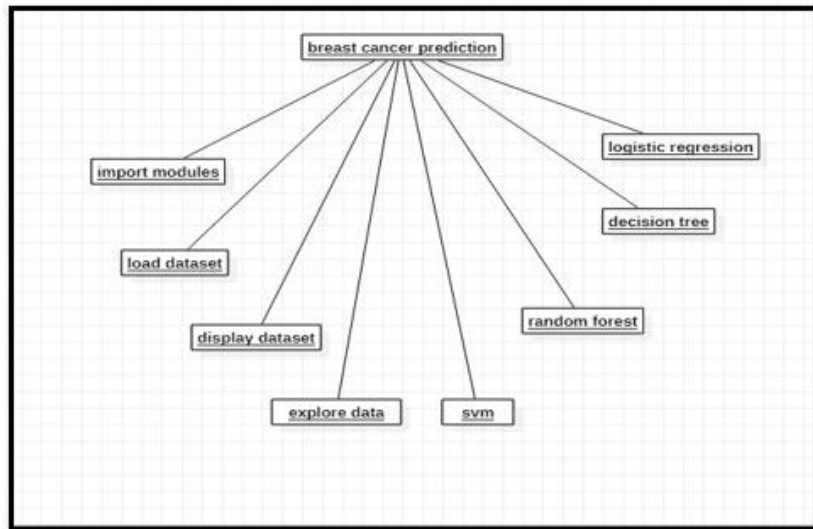
Examples Predictions: Using test data, the app will show some examples of predictions. Based on the chosen characteristics, this will display the tumour's anticipated categorization.



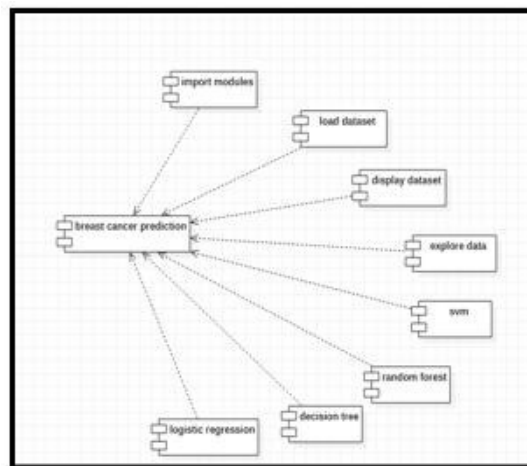
Class Diagram



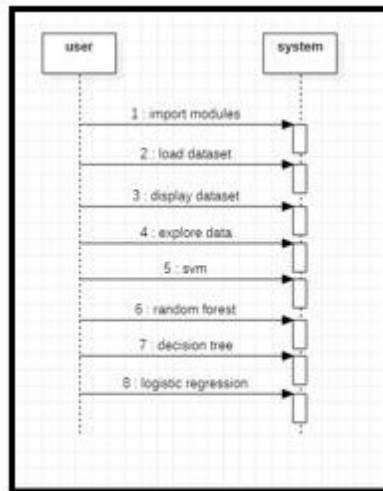
Object Diagram



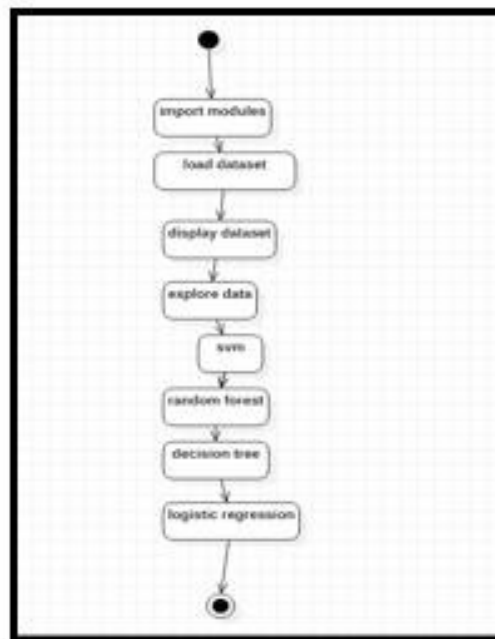
Component Diagram



Sequence Diagram



State Chart Diagram

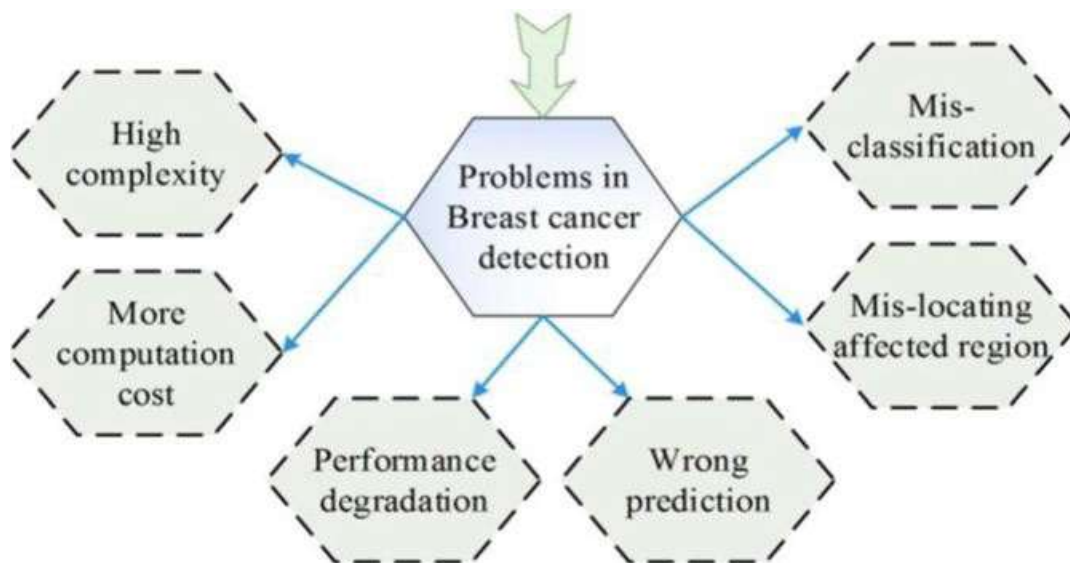


Shortcomings and Limitations

Dataset size: The Breast Cancer dataset offered by Scikit Learn is rather modest in size, and it could not accurately reflect real-world situations. The model's accuracy and capacity for extrapolating to unobserved data may be constrained as a result.

Feature Selection: The model is constructed utilising all of the characteristics present in the dataset, which may not be the best course of action. The performance of the model could be enhanced by feature selection methods like feature importance ranking or dimensionality reduction.

Model bias: The model may show bias towards particular characteristics or demographics in the data, which might have detrimental effects in real life. Any potential model biases must be carefully examined and eliminated.

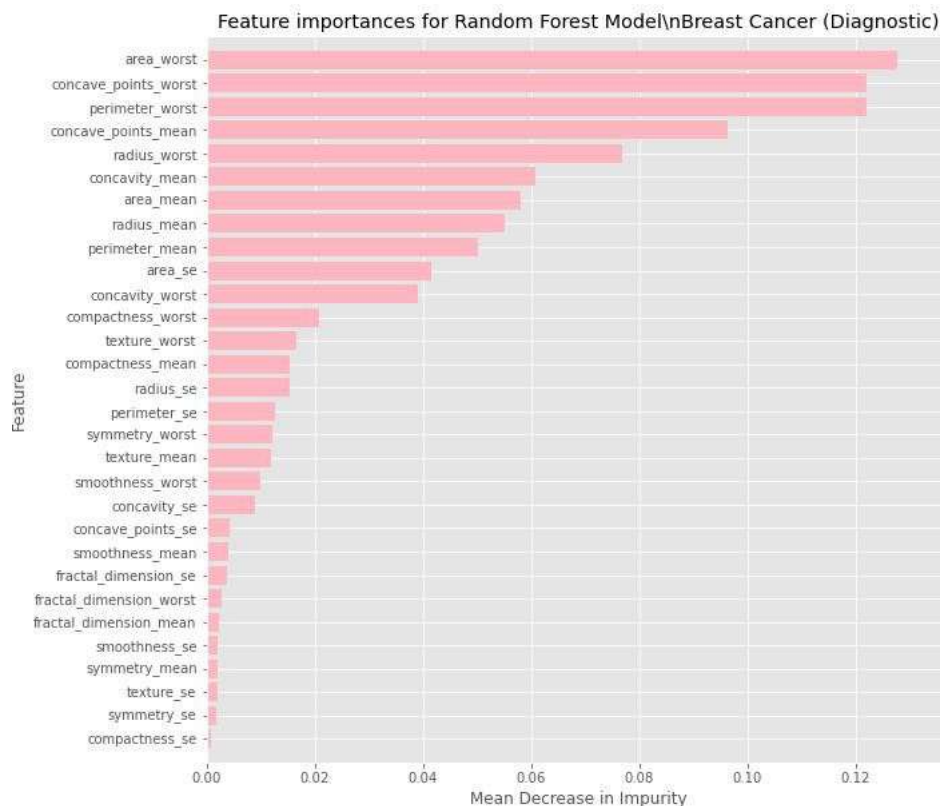


Chapter 4

Implementation

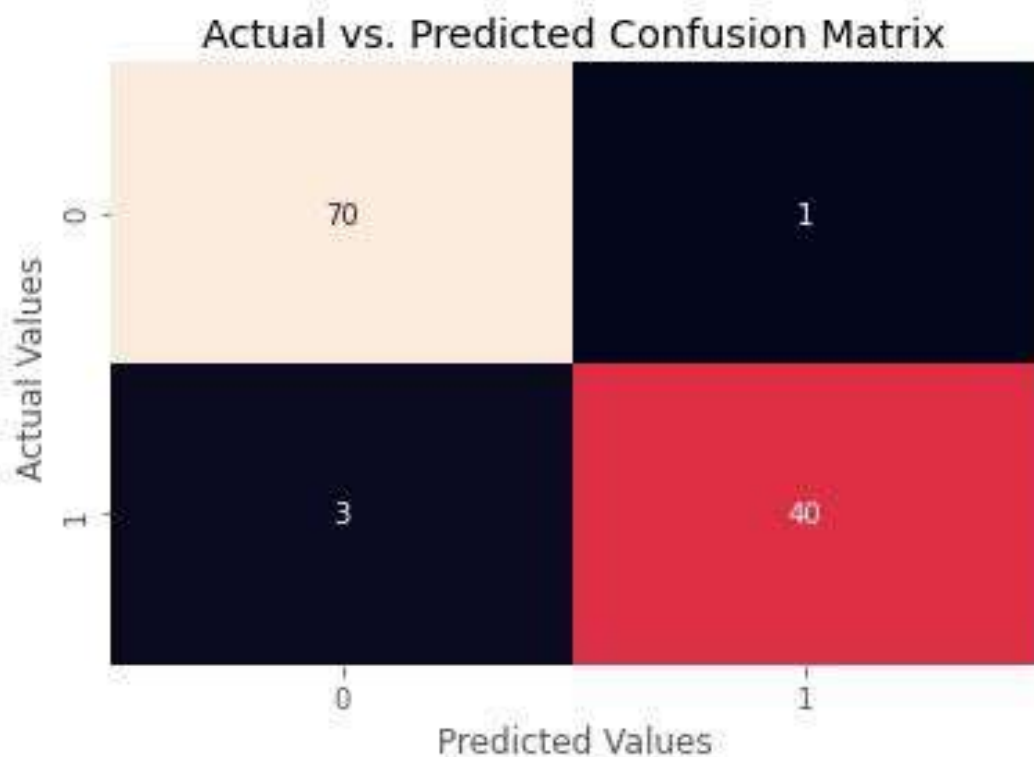
Data Collection and Preprocessing:

Gathering the data is the initial stage in developing this app. Since Scikit Learn already includes the Breast Cancer dataset, we can quickly import it into our Python code. Before we can utilise the dataset for classification, it must first go through preprocessing. Data cleansing, missing value removal, and feature scaling are all part of preprocessing. To clean and prepare the data, we may use the Pandas library, and to scale the features, we can use Scikit Learn's StandardScaler class.

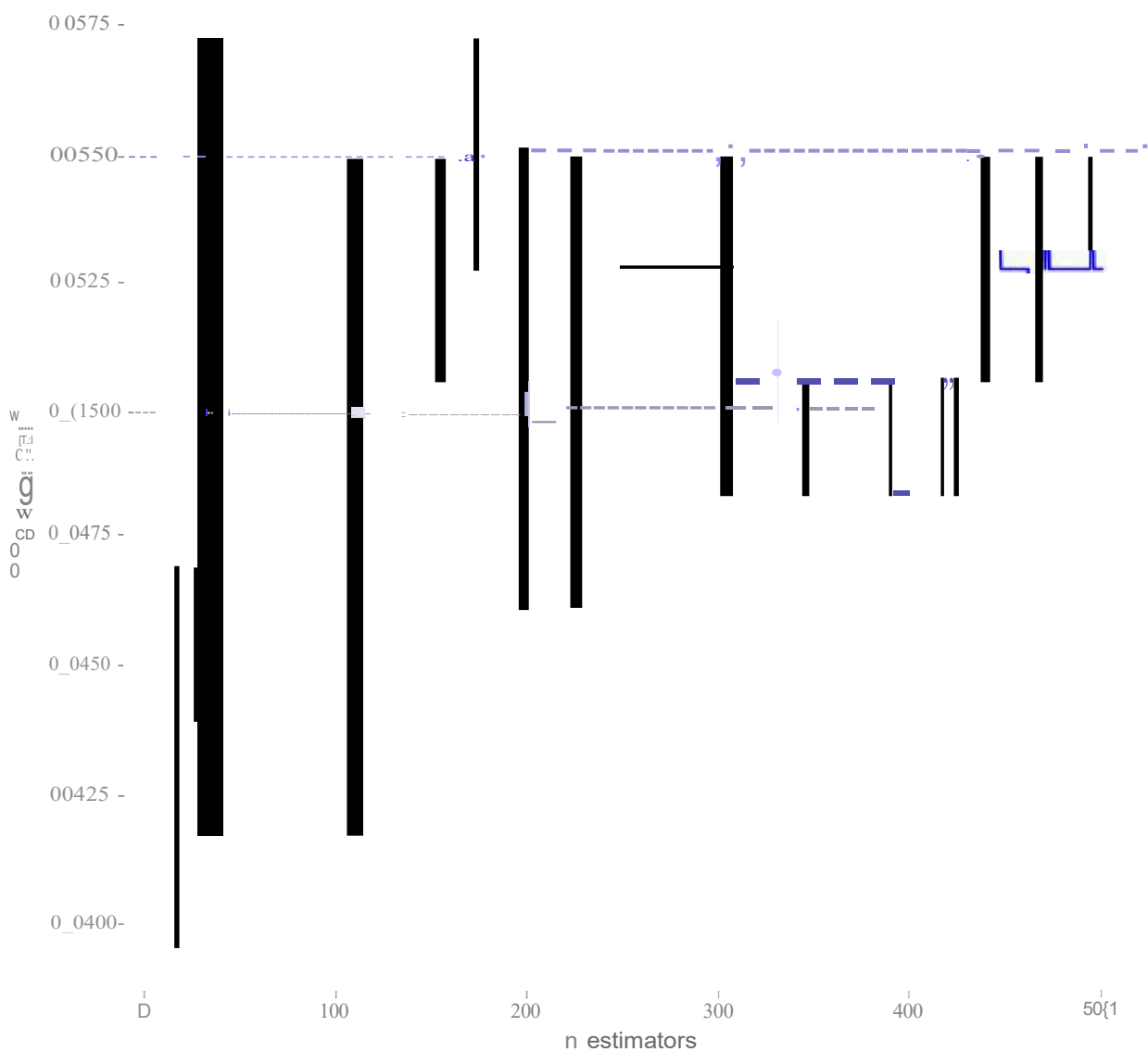


Data Visualization:

Our app's second page will be devoted to data visualisation. Any data science project must include data visualisation because it allows us to better understand the data and spot any patterns or trends. Plots of all kinds, including scatter plots, line plots, and bar plots, may be made using Seaborn. In addition, a 3D scatter plot may be used to show how three variables are related to one another. To see how all the variables in our dataset are correlated, we can also utilise a correlation map.



OOB Error Hate Across various Forest sizes
(Frnm 15 to 1000 trees}



Breast Cancer Classifier:

Our app's home page will have a breast cancer classification tool. To determine if the tumours are benign or malignant, we will categorise them using Scikit Learn's Random Forest Classifier. An ensemble learning system called the Random Forest Classifier mixes many decision trees to provide predictions. Because of the algorithm's excellent accuracy and resistance to overfitting, it is well recognised.

The user may modify the Random Forest Classifier's hyperparameters, such as the number of trees in the forest, the maximum depth of the trees, and the minimum amount of samples needed to divide an internal node, using Streamlit's slider widget. We may also show the classifier's accuracy and confusion matrix to assess its performance.

Terminology:

Our app's third segment will be devoted to terminology. It is crucial to have a section that clarifies the language used in the breast cancer dataset since medical terminologies might be perplexing. To make a user-friendly interface that shows the terminology and its description, we may utilise Streamlit's markdown widget.

Sidebar:

The sidebar section, which enables user interaction and offers more details about the app's functioning, is a crucial part of our app. In the context of our project, the sidebar may be used to show the Random Forest Classifier's hyperparameters and provide users with the option to change them using Streamlit's slider widget. The classifier's behaviour is controlled by the hyperparameters, which include the number of trees in the forest, the maximum depth of the trees, and the minimum amount of samples needed to divide an internal node.

Chapter 5

Standards Adopted

Coding Standards

The coding standards for this project include:

Python PEP 8: To maintain uniformity and readability throughout the codebase, the code should adhere to the Python PEP 8 coding style guide.

Docstrings: To offer clear and succinct documentation on their purpose and inputs/outputs, all functions and methods should provide docstrings.

Modularization: To encourage reusability, maintainability, and testing simplicity, the code should be divided into distinct functions and modules.

Code Comments: Code comments should be used to clarify intricate algorithms or sections of code that other developers might not understand right away.

Design Standards

The design standards for this project include:

Responsive Design: In order to make the web app usable and functioning across a variety of devices and screen sizes, it should be created with responsive design concepts in mind.

Consistency: For a smooth user experience, the web app's design needs to be the same on all pages and sections.

Usability: The web application should be made with a user-friendly design strategy to make it simple for users to engage with and comprehend its capabilities.

Accessibility: The online application has to be made accessible so that people with impairments may use it.

Testing Standards

The testing standards for this project include:

Unit testing: To make sure that each function and method works as intended, the code should be tested using unit testing frameworks like Pytest.

Integration testing: The various web app modules and parts should be tested in conjunction with one another to make sure they work together properly and achieve the intended outcome.

Functional Testing: It is important to check that the web application satisfies the functional specifications given in the project specifications.

User Acceptance Testing: To make sure the web app is user-friendly and satisfies users' wants and expectations, it should be put to the test by actual users.

Chapter 6

Conclusion and Future Scope

Conclusion

In this project, we used Python tools like Scikit-learn, Pandas, Seaborn, and Streamlit to construct a Streamlit web application. Using the random forest classifier, we classified tumours as benign or malignant using the built-in breast cancer dataset from Scikit-learn.

The programme became a multi-segment application after we included three segments, including Terminology, Data Visualisation, and Breast Cancer Classifier. On the introduction page's video, we also added an About section to the sidebar.

Additionally, we added sliders to the sidebar so that users could interactively change the random forest classifier's hyperparameters and observe the performance changes.

Future Scope

This project still has a great deal of room for expansion and enhancement. Here are some potential areas of focus:

Other Classification Models: We may test many classifiers to discover which one works better on the given dataset. Additionally, we can research ensemble methods and evaluate how well they perform in comparison to the random forest classifier.

Hyperparameter Tuning: To fine-tune the hyperparameters of the random forest classifier and raise accuracy, we may utilise more sophisticated methods like grid search or Bayesian optimisation.

Feature Selection: To find the most crucial characteristics and lessen the dataset's dimensionality, we may use feature selection techniques like PCA or Lasso.

We can upload this web application to a cloud computing platform like AWS or Google Cloud, making it accessible to a larger audience.

References

1. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Vanderplas, J. (2011). Scikit-learn: Machine learning in Python. *Journal of machine learning research*, 12(Oct), 2825-2830. [Link: <https://jmlr.csail.mit.edu/papers/volume12/pedregosa11a/pedregosa11a.pdf>]
2. Breast Cancer Wisconsin (Diagnostic) Data Set. University of California, Irvine Machine Learning Repository. [Link: [https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))]
3. McKinney, W., & others. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (Vol. 445, pp. 51-56). [Link: https://www.researchgate.net/profile/Wes-Mckinney/publication/265783370_Data_structures_for_statistical_computing_in_Python/links/5426f4210cf248c42ae1ff60/Data-structures-for-statistical-computing-in-Python.pdf]
4. Waskom, M., Botvinnik, O., O'Kane, D., Hobson, P., Lukauskas, S., Gemperline, D. C., ... & Augspurger, T. (2021). *mwaskom/seaborn: v0.11.1* (January 2021). Zenodo. [Link: <https://doi.org/10.5281/zenodo.4311642>]
5. Streamlit, Inc. (2021). Streamlit: The fastest way to build data apps. [Link: <https://streamlit.io/>]
6. Liaw, A., & Wiener, M. (2002). Classification and regression by random forest. *R news*, 2(3), 18-22. [Link: https://www.stat.berkeley.edu/~breiman/RandomForests/cc_home.htm]
7. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. [Link: <https://ieeexplore.ieee.org/document/4160265>]